# Checkpoint 3 - Análise Sintática com Geração da AST e Visitors

Parte 1 — Projeto da hierarquia de nós

Maria Rita Vieira Souza- 12321BCC024, Ana Clara Morais de Carvalho- 12311BCC048

**Link para o Github do projeto:** https://github.com/mariarvsouza/GBC071-Construcao-de-Compiladores

## 1 Alterações na gramática

Foram necessárias algumas alterações na gramática:

1. As produções do for foram alteradas para permitir a declaração da variável usada no laço e não apenas o ASSIGN.

2. Criação de produções para permitir a declaração de funções, o funcionamento correto dos parâmetros passados e a chamada de funções, e todas as alterações decorrentes disso para fatoração à esquerda e eliminação da recursão à esquerda.

3. Criação das produções dos literais booleanos, que não tinham sido contemplados na gramática original.

Todos os detalhes e a nova gramática final se encontram no pdf "gramatica-first-follow".

## 2 Hierarquia de nós

| **Categoria sintática** | **Nome do nó** | **Atributos** | **Produções geradoras** |
|---|---|---|---|

| | | | |
|---|---|---|---|
| Programa | ProgramNode | statements (lista de StmtNode) | Programa -> StmtList, StmtList -> Stmt StmtList \| $\epsilon$, Stmt -> Type "ID" Decl' \| Assign \| IfStmt \| ForStmt \| ReturnStmt \| BreakStmt \| Block |
| Declaração de Função | FuncDeclNode | returnType (String), name (String), params (lista de ParamNode), body (BlockNode) | Decl' -> "LPAREN" ParamList "RPAREN" Block \| VarDecl' |
| Parâmetro | ParamNode | type (String), name (String) | Param -> Type "ID", ParamList -> Param ParamList' \| $\epsilon$, ParamList' -> "COMMA" Param ParamList' \| $\epsilon$ |
| Bloco | BlockNode | statements (lista de StmtNode) | Block -> "LBRACE" StmtList "RBRACE" |
| Declaração de Variável | VarDeclNode | type (String), varID (String), initExpr (ExprNode) | Decl' -> "LPAREN" ParamList "RPAREN" Block \| VarDecl', VarDecl' -> "SEMICOLON" \| "ASSIGN" Expr "SEMICOLON" |
| Atribuição | AssignNode | varID (String), expr (ExprNode) | Assign -> AssignBase "SEMICOLON", AssignBase -> "ID" "ASSIGN" Expr |
| If/Else | IfStmtNode | condition (ExprNode), thenBlock (BlockNode), elseBlock (BlockNode) | IfStmt -> "KW_IF" "LPAREN" Expr "RPAREN" Block IfStmt', IfStmt' -> "KW_ELSE" Block \| $\epsilon$ |
| Repetição (For) | ForStmtNode | init (AssignNode / VarDeclNode), condition (ExprNode), increment (AssignNode), body (BlockNode) | ForStmt -> "KW_FOR" "LPAREN" ForInit "SEMICOLON" Expr "SEMICOLON" AssignBase "RPAREN" Block, ForInit -> Type "ID" "ASSIGN" Expr \| AssignBase |

| | | | |
|---|---|---|---|
| Retorno | `ReturnStmtNode` | `expr` (`ExprNode`) | `ReturnStmt -> "KW_RETURN" ReturnStmt', ReturnStmt' -> Expr "SEMICOLON" \| "SEMICOLON"` |
| Interrupção | `BreakStmtNode` | `--` | `BreakStmt -> "KW_BREAK" "SEMICOLON"` |
| Chamada de Função | `CallFuncExprNode` | `func` (String), `args` (lista de `ExprNode`) | `Atom -> "ID" Atom', Atom' -> "LPAREN" ArgList "RPAREN" \|` $\epsilon$, `ArgList -> Expr ArgList' \|` $\epsilon$, `ArgList' -> "COMMA" Expr ArgList' \|` $\epsilon$ |
| Expressão Unária | `UnaryExprNode` | `operator` (String), `operand` (`ExprNode`) | `UnaryExpr -> "NOT" UnaryExpr \| Atom` |

| | | | |
|---|---|---|---|
| Expressão Binária | `BinExprNode` | `left` (`ExprNode`), `operator` (String), `right` (`ExprNode`) | `OrExpr -> AndExpr OrExpr'`, `OrExpr' -> "OR" AndExpr OrExpr'` \| $\epsilon$, `AndExpr -> EqExpr AndExpr'`, `AndExpr' -> "AND" EqExpr AndExpr'` \| $\epsilon$, `EqExpr -> RelExpr EqExpr'`, `EqExpr' -> "EQ" RelExpr EqExpr'` \| `"NE" RelExpr EqExpr'` \| $\epsilon$, `RelExpr -> AddExpr RelExpr'`, `RelExpr' -> "GT" AddExpr RelExpr'` \| `"GE" AddExpr RelExpr'` \| `"LT" AddExpr RelExpr'` \| `"LE" AddExpr RelExpr'` \| $\epsilon$, `AddExpr -> MultExpr AddExpr'`, `AddExpr' -> "PLUS" MultExpr AddExpr'` \| `"MINUS" MultExpr AddExpr'` \| $\epsilon$, `MultExpr -> UnaryExpr MultExpr'`, `MultExpr' -> "MULT" UnaryExpr MultExpr'` \| `"DIV" UnaryExpr MultExpr'` \| `"MOD" UnaryExpr MultExpr'` \| $\epsilon$ |
| Identificador | `IdExprNode` | `id` (String) | `Atom -> "ID" Atom'`, `Atom' -> "LPAREN" ArgList "RPAREN"` \| $\epsilon$ |
| Literal Inteiro | `IntLitNode` | `value` (int) | `Atom -> "NUMBER_LIT"` |
| Literal Real | `RealLitNode` | `value` (float / double) | `Atom -> "REAL_LIT"` |
| Literal Caractere | `CharLitNode` | `value` (char) | `Atom -> "CHAR_LIT"` |
| Literal String | `StringLitNode` | `value` (String) | `Atom -> "STR_LIT"` |

| | | | |
|---|---|---|---|
| Literal Booleano | `BoolLitNode` | `value` (boolean) | `Atom -> "TRUE_LIT" \| "FALSE_LIT"` |

# 3 Mapeamento de gramática para árvore

## 3.1 Não-terminais que viraram nós da AST

Programa (ProgramNode), FuncDecl (FuncDeclNode), Param (ParamNode), Block (BlockNode), VarDecl (VarDeclNode), Assign (AssignNode), IfStmt (IfStmtNode), ForStmt (ForStmtNode), ReturnStmt (ReturnStmtNode), BreakStmt (BreakStmtNode) e CallFuncExpr (CallFuncExprNode).

## 3.2 Não-terminais que viraram atributos de outros nós

Type se torna o atributo type armazenado no VarDeclNode.

AssignBase é subentendido nos atributos de AssignNode e ForStmtNode.

StmtList se torna uma lista de comandos "statements (lista de StmtNode)"dentro de ProgramNode e BlockNode.

## 3.3 Não-terminais que não se tornaram nada

Não-terminais criados para manter a precedência dos operadores (Expr, OrExpr, AndExpr, EqExpr, RelExpr, AddExpr, MultExpr), já que na AST a precedência é representada diretamente na árvore.

Não-terminais criados para eliminar a recursão à esquerda (OrExpr', AndExpr', EqExpr', RelExpr', AddExpr', MultExpr'), já que foram criados apenas para impedir os loops infinitos no parser descendente.

Não-terminais criados pela fatoração à esquerda (VarDecl', IfStmt', ReturnStmt'), já que foram criados apenas para permitir a escolha entre produções com o mesmo prefixo usando só um token de lookahead.

Delimitadores (LPAREN, RPAREN, COMMA, SEMICOLON, LBRACE, RBRACE) também não viram nós na AST.

# 4 Posição

Todo nó da AST herda de uma classe abstrata ASTNode que captura os atributos line e col a partir do primeiro token que deu origem ao nó. A análise semântica vai reportar

erros com essa posição, e é muito mais barato guardá-la agora do que voltar atrás depois.

# 5 Árvore

**Programa exemplo da linguagem para a árvore:**

```
number process (number x) {
    if (x>0) {
        for (number i=0; i<x; i=i+1) {
            x = x-1;
        }
    }
    return x;
}
```

**Pedaços que a árvore descartou:**

1. Na árvore, para montar a expressão "x>0", por exemplo, é possível utilizar o `BinExprNode` (com `IdExprNode` e `IntLitNode`), descartando toda a hierarquia de derivação dos operadores que é usada para manter a precedência (`Expr -> OrExpr -> AndExpr -> EqExpr -> RelExpr -> AddExpr -> MultExpr -> UnaryExpr -> Atom -> "ID"`).

2. Os tokens delimitadores, pontuações e palavras-chave sintáticas (`"KW_IF"`, `"KW_FOR"`, `"LPAREN"`, `"RPAREN"`, `"LBRACE"`, `"RBRACE"`, `"SEMICOLON"`) foram descartados.

3. Os não-terminais auxiliares introduzidos para fatoração à esquerda e para eliminação de recursão à esquerda foram descartados.

**Representação usada na impressão do código:**

```
ProgramNode
  FuncDeclNode(returnType: "number", name: "process")
    ParamNode(type: "number", name: "x")
    BlockNode
      IfStmtNode
        BinExprNode(operator: ">")
          IdExprNode(id: "x")
          IntLitNode(value: 0)
        BlockNode
          ForStmtNode
            VarDeclNode(type: "number", varID: "i")
              IntLitNode(value: 0)
            BinExprNode(operator: "<")
              IdExprNode(id: "i")
              IdExprNode(id: "x")
            AssignNode(varID: "i")
```

```
        BinExprNode(operator: "+")
          IdExprNode(id: "i")
          IntLitNode(value: 1)
      BlockNode
        AssignNode(varID: "x")
          BinExprNode(operator: "-")
            IdExprNode(id: "x")
            IntLitNode(value: 1)
ReturnStmtNode
  IdExprNode(id: "x")
```

**Árvore do programa feita à mão antes da implementação em código:**

